Loscalzo Lab -
Brigham and Women's hospital

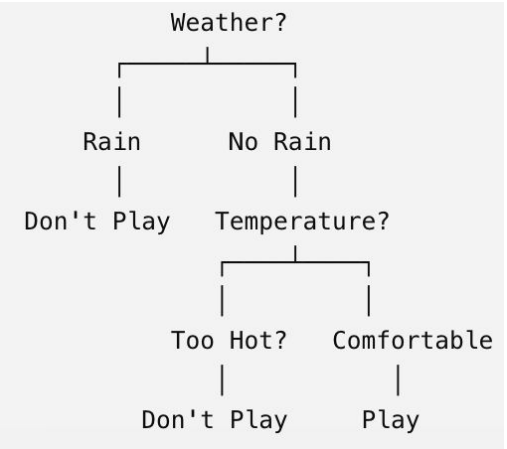
Session 2 - Trees, Forests, and Boosting

Classical Machine Learning Workshop
by Jonah Sandow

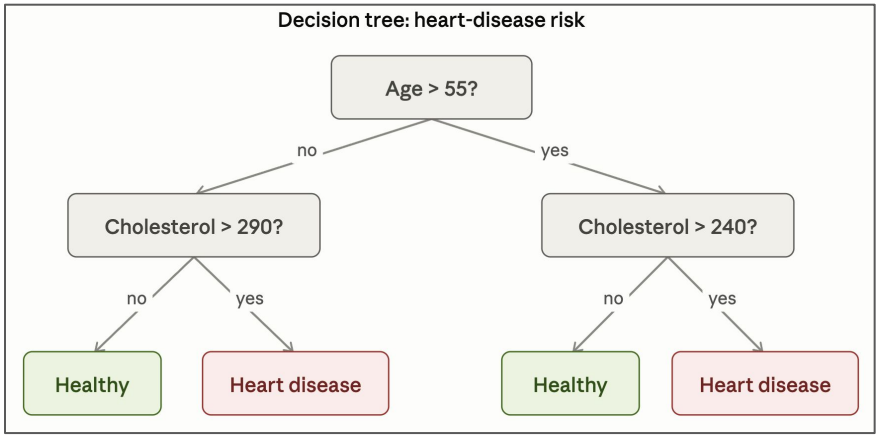
Decision trees

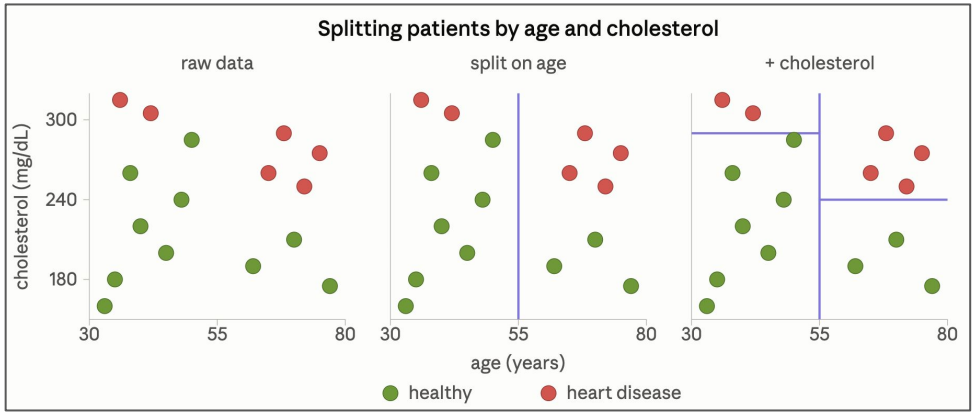
- You already use decision trees everyday, you just don't call it that.
- "Should I go grocery shopping?" -> you ask small yes/no questions: *Is the grocery store Open? Do I already have food at home? Do I need to cook soon?*
    - Each Answer sends you down a path until you land on a decision
- A decision tree model works the same way
    - a chain of yes/no questions that ends in a prediction
- Advantage: you can read the tree and see exactly why it made each prediction (nothing is hidden)

Deciding whether or not to play a sport



Heart-disease Risk Example





- Each question in the tree (left) is one straight cut in the scatter plot (right)

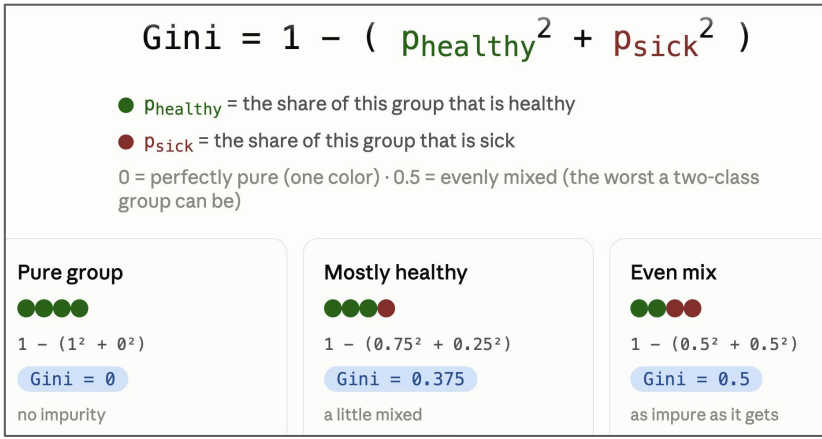
Takeaway:
The model is the lines/splits (the questions the tree learned).

How a Tree Is Built?

- The model isn’t handed the question and the thresholds. It finds them in the data.
- It looks at each feature (age, cholesterol) and each possible threshold -> And picks the one that best separates the 2 classes (sick and healthy)
- Then it repeats this inside each of the new groups with the other feature.
- It stops when the groups are pure enough or it runs out of features

Loss function - Gini impurity

- A common loss function for Decision trees is The Gini Impurity
  - Remember, our goal in training is to minimize the loss function so The tree picks the cut that **lowers impurity the most**.
- Gini Impurity works by calculating how pure each bucket is:
  - 0 = all one color (pure), 0.5 = a 50/50 mix (as mixed as it gets).
- Because we have multiple buckets at the end, we take the weighted average of the Gini impurities from each bucket and try to minimize that



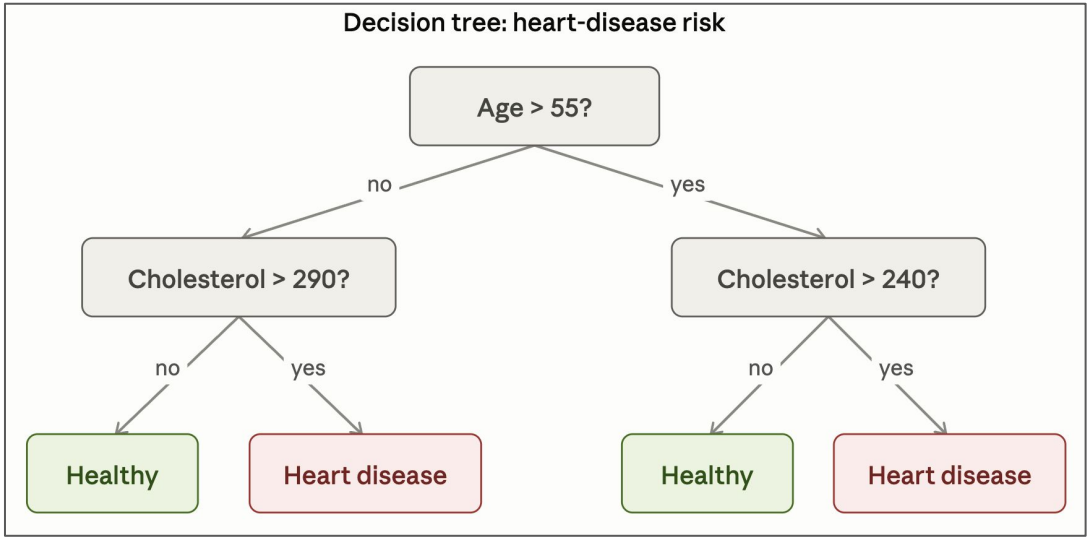
* You don't have to compute this by hand, the algorithm tries the cuts and keeps the lowest-impurity one.

You just need the intuition: lower Gini = cleaner split.

Comprehension Check

Does the Decision tree predict these patients to be sick or healthy?

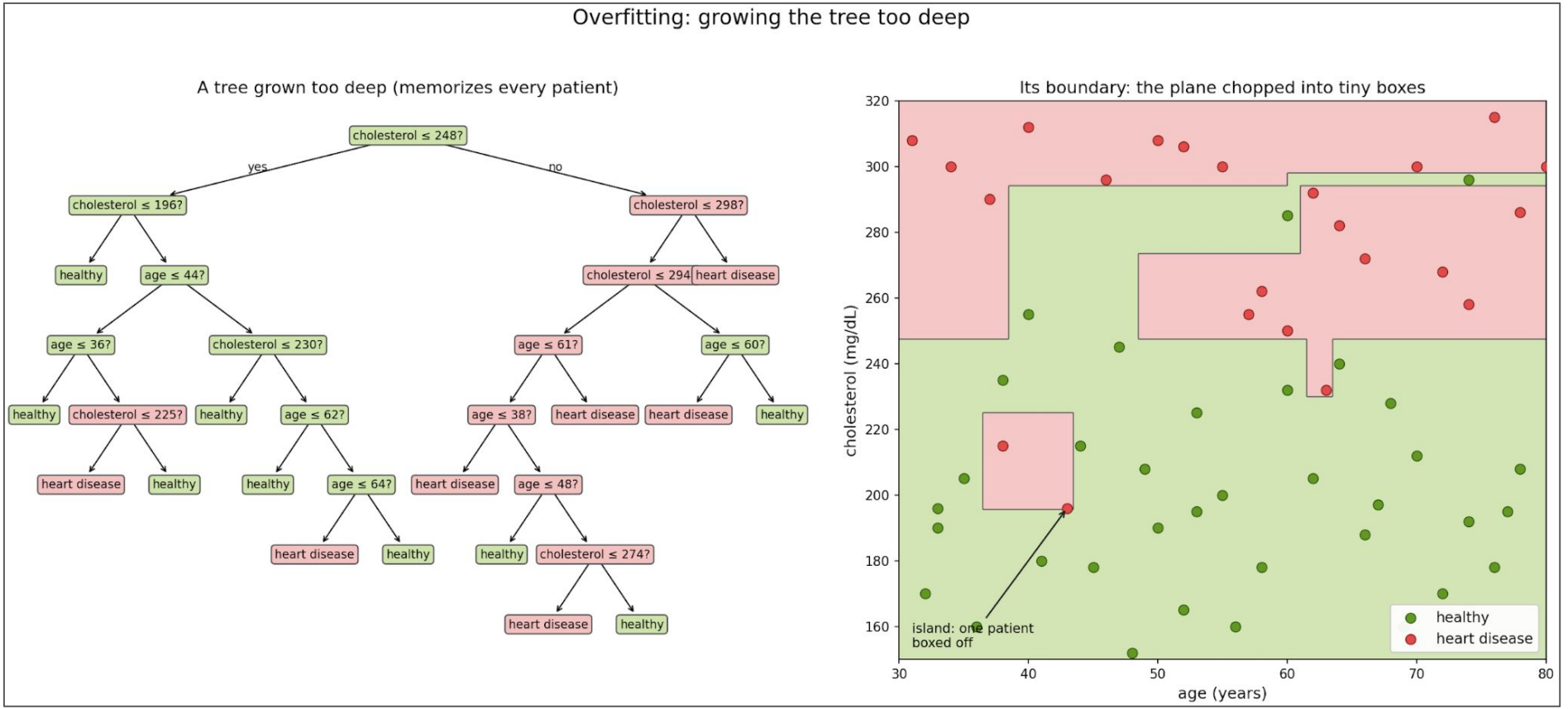
1. Patient A - age 68, cholesterol 265 — Sick
2. Patient B - age 45, cholesterol 305 — Sick
3. Patient C - age 40, cholesterol 260 — Healthy



Trees Can Overfit

- Imagine the tree keeps asking questions until every single patient (or sample) sat in its own perfectly pure box.
    - That would be Overfitting
- This Happens when we grow a tree too deep
- A too-deep tree memorizes the training patients instead of learning the general pattern
    - It looks perfect on data it's seen and fails on new patients.

Growing a tree too deep



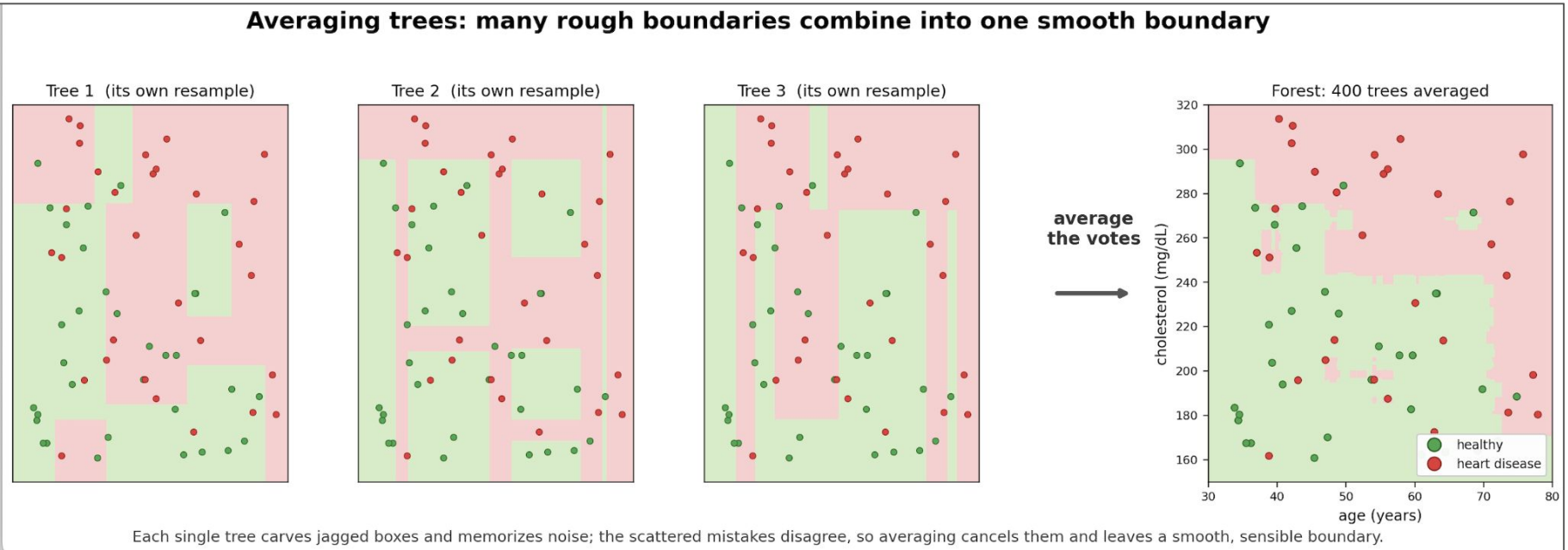
Comprehension Check

Let's say there's a tree with 200 splits for 200 patients and 100% training accuracy. Should we trust it on a new patient? Why or why not?

No, it’s probably overfitting. We should try it out on new test data first

Random forests

- Don’t trust one tree, grow **many** trees and let them vote
  - For a new patient, each tree says sick or healthy, and we take the majority.
- One trees weird mistakes get outvoted by the others. The Shared signal survives.

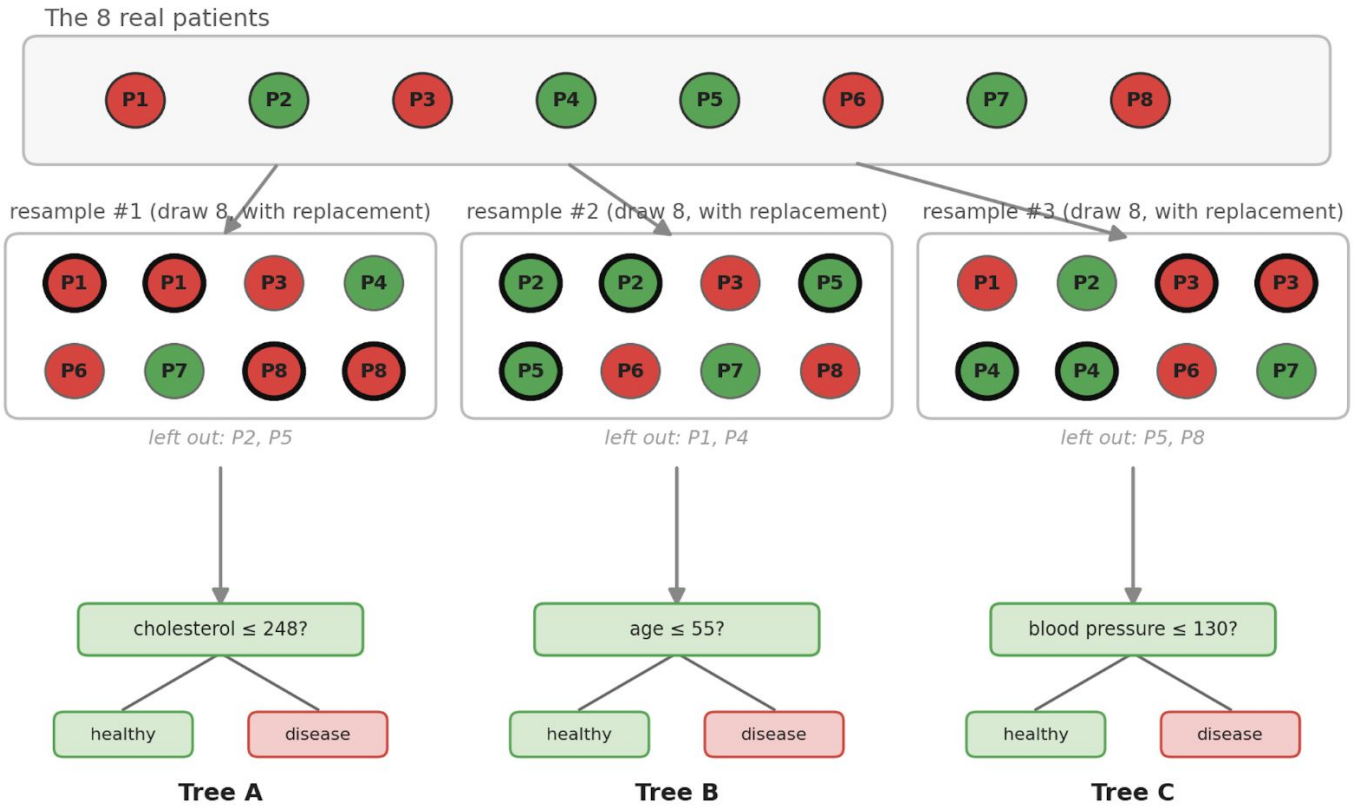


Bagging

- Problem 1: If every tree is trained on the same data, they all come out identical.
- Bagging = giving each tree its own random resample of the patients, drawn with replacements (some samples appear twice, some not at all)
    - Lets say we have N patients in our training set. To build one tree's training set, we draw N times from the patient list, and each draw is uniformly random over all N patients with the drawn patient put back each time
- Each tree grows a different shape (to different questions) and makes different mistakes.
- Bonus: Run each patient through only the trees that didn't train on it -> the 'out-of-bag' score, a free accuracy estimate with no separate test set needed.

Bagging: each tree trains on its own random resample of the patients

bold outline = a patient drawn more than once



Nobody picks which patients repeat — the random draw decides, fresh for every tree. ≈63% of patients land in each resample; the rest are “out-of-bag.”

Feature shuffling

- Problem 2: Even with resampling (bagging) if one feature is very strong (ex: cholesterol), almost every tree will split on it first - so the trees will still look similar.
- Fix: At each split let the tree only choose from a random handful of features, not all of them. Sometimes cholesterol isn't in a tree's options so it has to learn patterns from other features.
  - We pick a number *m* < total number of features. At each node before choosing a split the tree grabs a random *m* features from the total feature list and is only allowed to split on one of those. Next node repeats this.
  - A common *m* for classification is the square root of the total feature count.
- This makes the trees even more different from each other - they each learn from different features, making averaging them pay off.

Feature shuffling: at each split the tree may only choose from a random handful of features

the random handful is re-drawn fresh at every node — a common size is m = √(total features), e.g. 4 of 16

A plain tree — all features available

features available at this split:

- **cholesterol**
- age
- blood pressure
- max heart rate

↓

split on cholesterol

healthy | disease

cholesterol is the strongest signal, so almost every tree grabs it first

A forest tree — random subset at this split

features available at this split:

- ~~cholesterol~~
- **age**
- blood pressure
- ~~max heart rate~~

↓

split on age

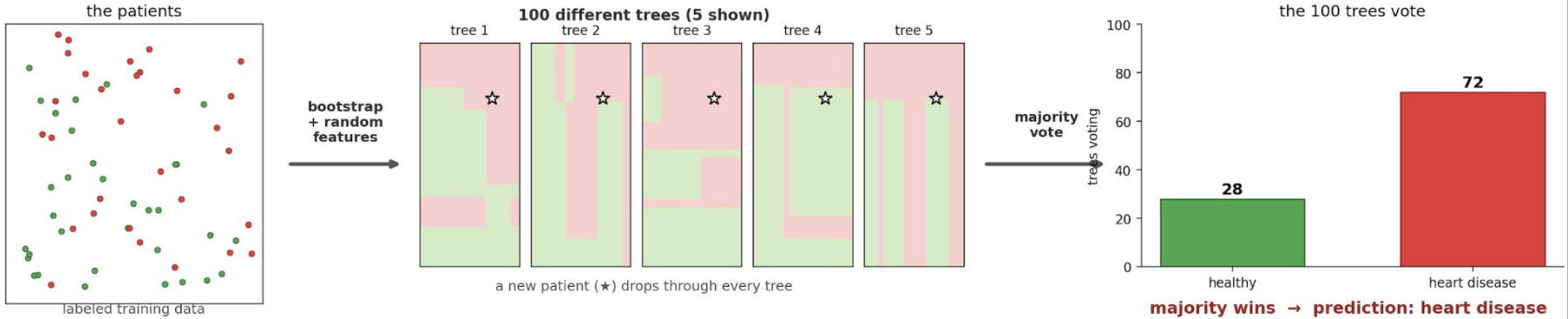
healthy | disease

cholesterol wasn't drawn this time, so the tree is forced to split on age

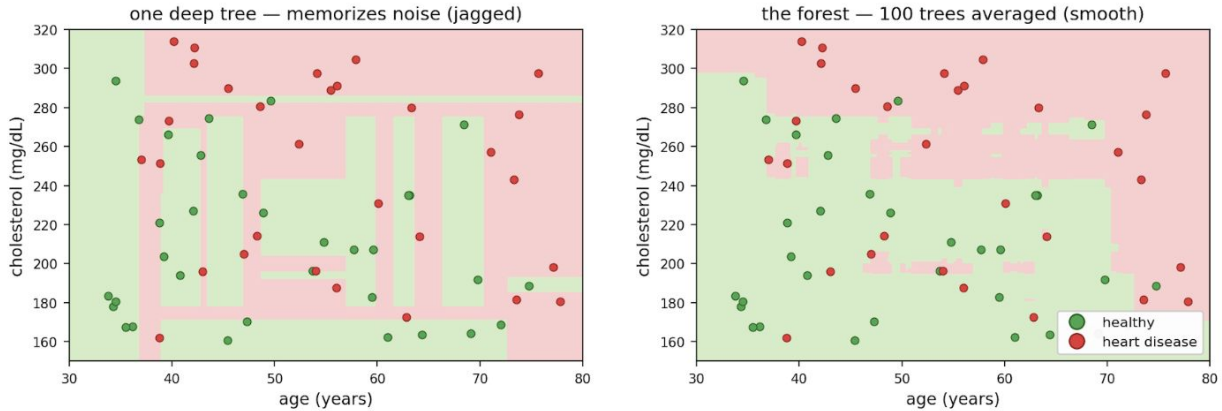
= chosen split | = offered | = not on the menu this split

Random forest: build many different trees, then let them vote

a crowd of deliberately different trees — each on its own resample and its own random features — outvotes any single tree's mistakes



The payoff: the forest is the clean version of one overfit tree



Comprehension Check

1. If I train 100 trees on the exact same data with no randomness, what does the forest predict versus one tree? and why is that useless?

   They predict the same. With no randomness, all 100 trees are identical, so they all vote the same way. Averaging 100 copies of one opinion is still just that one opinion. You need the trees to differ.

2. Hiding cholesterol from a tree can only make that tree worse at its job. So why does forcing every tree to sometimes ignore the strongest feature make the whole forest better?

   It allows trees to learn other patterns in the data that they wouldn't see if they only looked at the most prominent pattern.

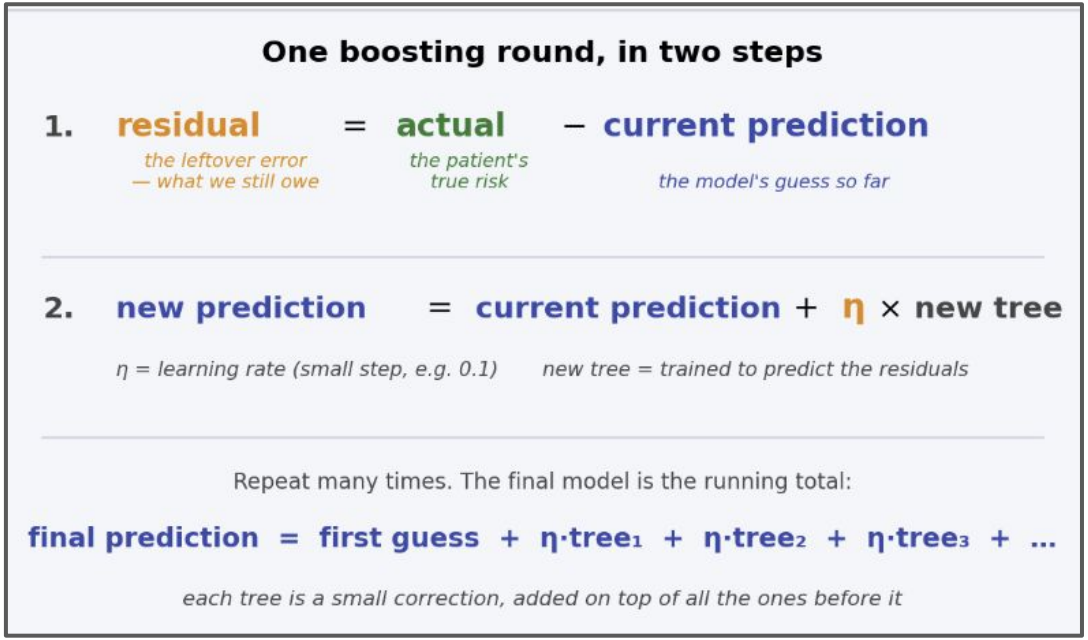
Gradient Boosting

- Another way to combine decision trees
- Builds many **small decision trees sequentially** (instead of all at once like forests)
- Each new small tree learns from the mistakes of the previous trees
- Trees are combined into one strong predictive model, where error drops a little more each round

Key Takeaway:
Instead of building one complex tree, build many simple trees that each fix what's still wrong.

Training

1. Train a small decision tree
2. Measure the error (residuals)
3. Train a new tree to predict those errors
4. Add the new tree's predictions to the existing model
5. Repeat steps 2-4 many times



Intuition

1 The dataset — inputs (X) and the value we predict (y)

The 3 training houses have known prices. The test house price is hidden during training — we only use it at the end to check our answer.

	X — features (inputs)				y — target
House	Size	Bed	Age	Dist	Price
1	2500	4	5	2	590k
2	1800	3	40	15	330k
3	3200	5	2	1	610k
test	2400	4	8	3	580k ← test house

Rows 1–3 = training set (model learns from these). Bottom row = test set (its 580k is kept hidden until we check the prediction in step 3).

2 Training — build the trees one at a time, each predicting the last tree's error

A tree makes guesses; the leftover error is what it missed (target – guess). That leftover error becomes the next tree's target.

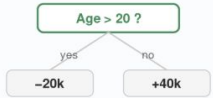
Tree 1
target (y) = the actual price

leftover = actual price – guess

target		guess		error left
590	–	560	=	+30
330	–	350	=	–20
610	–	560	=	+50

these errors become Tree 2's target

Tree 2
target (y) = Tree 1's errors (+30, –20, +50)

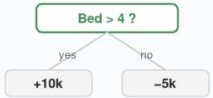


leftover = Tree 1's error – guess

target		guess		error left
+30	–	+40	=	–10
–20	–	–20	=	0
+50	–	+40	=	+10

these errors become Tree 3's target

Tree 3
target (y) = Tree 2's errors (–10, 0, +10)



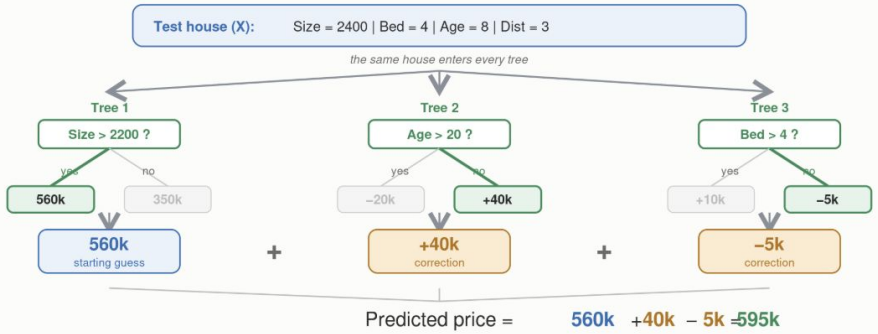
leftover = Tree 2's error – guess

target		guess		error left
–10	–	–5	=	–5
0	–	–5	=	+5
+10	–	+10	=	0

errors now small → stop adding trees

3 Prediction — run the test house through every tree, then add the outputs

Start with Tree 1's guess, then add each tree's correction. The sum is the final prediction.



Actual price was 580k — our prediction of 595k is off by only 15k.

Forests vs Boosting

- Forest: trees built independently, all at once, then averaged/voted. Order doesn't matter.
- Boosting: trees built in a chain, each fixing the last. Order is everything.
- Forest mainly smooths out noise; adding trees rarely hurts.
- Boosting chases down remaining error; adding too many trees can overfit, so it needs tuning (number of trees, learning rate, tree depth).

Thank You!
Any Questions?

Special thanks to Dr. Hau Xu, Marianna Lamanna, and Dr. Joseph Loscalzo for project guidance.